

AI Testing & RAG Playbook

A Guide to InsightIQ

Meridian Quality Systems
Product Documentation · Version 1.6 · Advanced Guide

About This Document

Difficulty	★★★ Advanced
Best for	Embeddings, Retrieval Strategy Comparison, Prompt Construction, Complex Parsing
Approximate Chunks	70
Estimated Embeddings	70
Recommended RAG Stages	Embedding → Retrieval → Prompt Construction → Parsing

This document introduces figures, diagrams, cross-references, and appendices, and is designed to demonstrate why parsing quality has an outsized effect on downstream retrieval.

Table of Contents

1. Introduction to InsightIQ
2. RAG Architecture Overview
3. Embeddings
4. Retrieval Strategies
5. Prompt Construction
6. Evaluation Metrics
7. Parsing Considerations
8. Deployment Patterns
9. Case Study: Documentation Search
10. Best Practices
- Appendix A — Extended Glossary
- Appendix B — Sample End-to-End Configuration
- References
- Questions to Try

1. Introduction to InsightIQ

InsightIQ is Meridian Quality Systems' platform for constructing, tuning, and evaluating Retrieval-Augmented Generation pipelines. Where FalconTest and APIVerify test conventional software behavior, InsightIQ exists because RAG systems fail in ways that conventional test suites are not built to catch: a RAG pipeline can pass every unit test and still return confidently wrong answers if retrieval surfaces the wrong context, or if a prompt template buries the retrieved context in a way the underlying model ignores.

InsightIQ was built by the same internal team that maintains this documentation suite, and — in a detail worth noting for anyone testing retrieval on this very document — InsightIQ is the system MQS uses internally to index and search its own FalconTest, APIVerify, and Engineering Standards documentation. That means the platform described in this playbook is, among other things, describing itself.

1.1 Scope of This Playbook

This playbook covers four pipeline stages in depth — Embeddings (Section 3), Retrieval (Section 4), Prompt Construction (Section 5), and Evaluation (Section 6) — and one supporting concern, Parsing (Section 7), which earlier documents in this suite deliberately avoided testing in depth. See Figure 1 for how these stages relate to one another end to end.

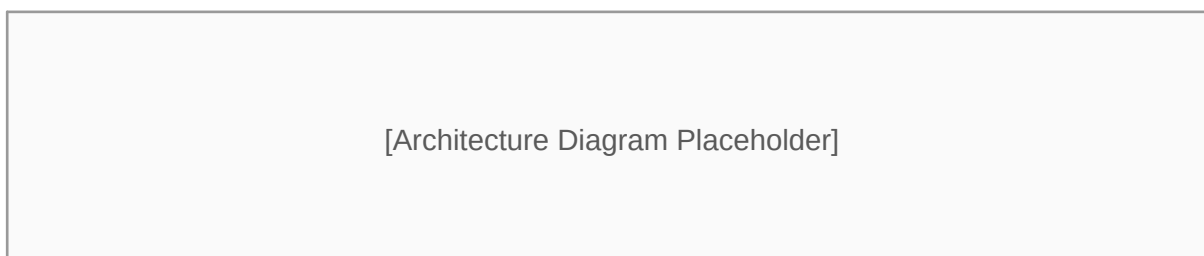


Figure 1: End-to-end InsightIQ pipeline, from raw document to generated answer.

2. RAG Architecture Overview

An InsightIQ pipeline is composed of five stages, executed in order for indexing and a subset of them again at query time.

Stage	Runs at Index Time	Runs at Query Time	Output
Parsing	Yes	No	Structured text and tables
Chunking	Yes	No	Fixed or variable-size text chunks
Embedding	Yes	Yes (for the query)	Dense vector representations
Retrieval	No	Yes	Ranked list of candidate chunks
Generation	No	Yes	Final natural-language answer

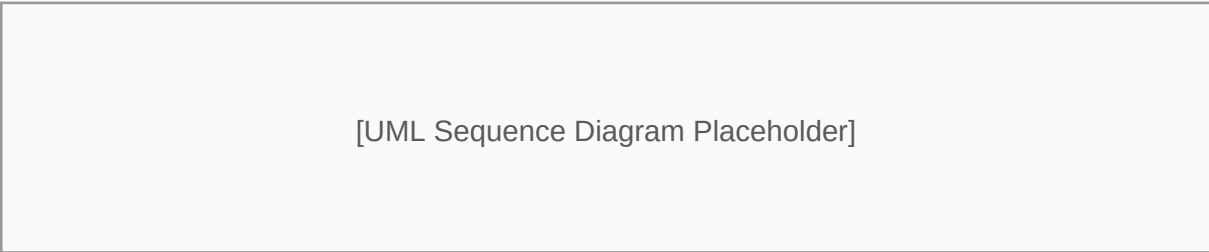


Figure 2: Sequence diagram showing index-time and query-time flows through the InsightIQ pipeline.

2.1 Index-Time vs. Query-Time

A common point of confusion for teams new to InsightIQ is assuming Embedding happens only once, at index time. In fact, Embedding runs twice in the lifecycle of a single query: once for every chunk at index time, and once more for the incoming query itself at query time, so that the query's vector can be compared against the chunk vectors already stored in the vector index. See Section 3.3 for why using the same embedding model for both is a hard requirement, not a recommendation.

3. Embeddings

InsightIQ ships with two supported embedding models.

Model	Dimensions	Relative Speed	Relative Accuracy	Recommended Use
VectorCore-S	384	Fast	Good	Local dev, small corpora (<10,000 chunks)
VectorCore-L	1024	Slower	Higher	Production, large corpora

3.1 VectorCore-S

VectorCore-S is a compact embedding model optimized for fast, local iteration. It produces 384-dimensional vectors and can embed roughly 2,000 chunks per second on a single CPU core, making it the default for InsightIQ's local development mode, where fast feedback matters more than squeezing out the last few points of retrieval accuracy.

3.2 VectorCore-L

VectorCore-L produces 1024-dimensional vectors and is meaningfully more accurate on InsightIQ's internal benchmark suite, at roughly one-third the throughput of VectorCore-S. It is the recommended model for any production deployment, and is the model InsightIQ itself uses to index the MQS documentation suite that this playbook is part of.

3.3 Embedding Model Consistency

A vector index built with one embedding model cannot be meaningfully queried using vectors from a different embedding model, because different models place semantically similar text at different coordinates in different, incompatible vector spaces. InsightIQ enforces this: the embedding model used to build an index is recorded in the index's metadata, and InsightIQ will refuse to run a query against an index using a different model than the one it was built with, raising a `ModelMismatchError` rather than silently returning meaningless results.

WARNING: Switching an InsightIQ project from VectorCore-S to VectorCore-L (for example, when moving from local development to production) requires a full re-index of the corpus. There is no supported migration path that reuses VectorCore-S vectors with VectorCore-L.

3.4 Chunk Size and Embedding Quality

Chunk size, set during the Chunking stage described in Section 2, has a direct effect on embedding quality. Chunks that are too small lose surrounding context, causing their embeddings to be ambiguous. Chunks that are too large mix multiple topics into a single vector, causing their embeddings to be diluted and less discriminative for any single topic within them. InsightIQ's internal benchmarking, run against the same ten-document corpus that includes this document, found that chunks between 400 and 700 characters produced the best retrieval accuracy for VectorCore-L; results for VectorCore-S followed a similar pattern but peaked slightly lower, between 350 and 600 characters.

4. Retrieval Strategies

InsightIQ supports three retrieval strategies, which can be used individually or combined.

4.1 Dense Retrieval

Dense Retrieval compares the query's embedding vector against every chunk's embedding vector using cosine similarity, returning the top-K most similar chunks. It is the default strategy and captures semantic similarity well — a query about "adjusting how often failed tests are re-run" can match a chunk about "Smart Retry" even without any shared keywords, because the embeddings capture meaning rather than exact wording.

4.2 Keyword Retrieval

Keyword Retrieval uses a traditional inverted-index term-matching approach (specifically, BM25 scoring) and is included primarily as a fallback for queries containing exact identifiers — configuration keys, error codes, or product names — that a purely semantic match might rank too low. See Figure 3 for a visual comparison of how Dense and Keyword retrieval rank the same query differently.

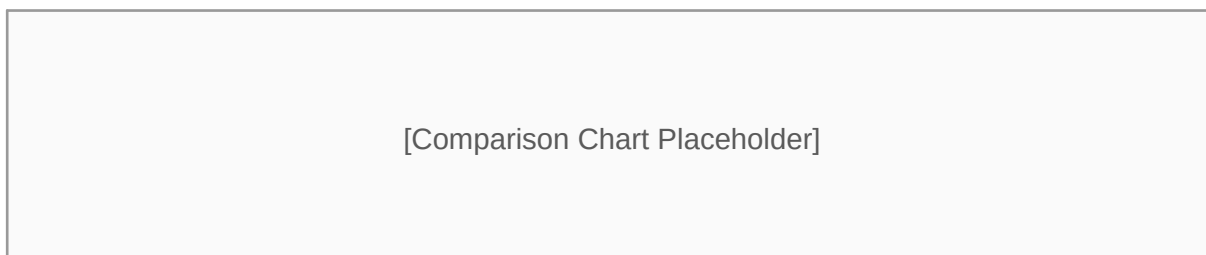


Figure 3: Ranking comparison between Dense Retrieval and Keyword Retrieval for the same sample query.

4.3 Hybrid Retrieval

Hybrid Retrieval runs both Dense and Keyword retrieval in parallel and merges their ranked results using a weighted combination, controlled by a single tunable parameter, `hybrid_alpha`, ranging from 0 (pure keyword) to 1 (pure dense). InsightIQ's default value for `hybrid_alpha` is 0.7, favoring dense retrieval while still giving keyword matches meaningful influence.

4.4 Choosing a Retrieval Strategy

Strategy	Strength	Weakness	Best For
Dense	Captures semantic meaning	Can miss exact identifiers	Conceptual, natural-language queries
Keyword	Exact identifier matching	Misses paraphrased queries	Config keys, error codes, product names
Hybrid	Balances both	Requires tuning <code>hybrid_alpha</code>	General-purpose production use

TIP: Start every new InsightIQ project with Hybrid Retrieval at the default `hybrid_alpha` of 0.7, and only move to a pure strategy if evaluation (Section 6) shows a clear, consistent advantage for your specific corpus and query patterns.

4.5 Re-Ranking

An optional Re-Ranking pass can be applied after any of the three strategies above. Re-Ranking uses a smaller, more computationally expensive model to re-score the initial top-K candidates (typically top-20) and return a refined top-N (typically top-4 to top-6) for the Prompt Construction stage described next in Section 5. Re-Ranking is disabled by default because of its added latency, and is recommended primarily for InsightIQ deployments where answer quality matters more than response time — see Section 8 for how this tradeoff is typically decided per deployment pattern.

5. Prompt Construction

Once Retrieval has returned a ranked list of candidate chunks, InsightIQ assembles them into a prompt for the generation model.

5.1 Prompt Template Structure

An InsightIQ prompt template has three parts, always assembled in this order: a system instruction, the retrieved context, and the user's original query.

```
[SYSTEM INSTRUCTION]
You are a documentation assistant. Answer only using the provided context.
If the answer is not in the context, say so explicitly.

[RETRIEVED CONTEXT]
{{ for chunk in retrieved_chunks }}
Source: {{ chunk.source }} (page {{ chunk.page }})
{{ chunk.text }}
{{ endfor }}

[USER QUERY]
{{ query }}
```

5.2 Context Window Budgeting

Every generation model InsightIQ supports has a maximum context window, and the retrieved chunks assembled in Section 5.1 must fit within it alongside the system instruction and query. InsightIQ budgets the context window using a fixed allocation: 10% reserved for the system instruction, 70% for retrieved context, and 20% for the query and generated response combined. If the retrieved chunks from Section 4 exceed the 70% context budget, InsightIQ truncates from the lowest-ranked chunk upward, never from the highest-ranked chunk downward, since the highest-ranked chunk is assumed to be the most relevant to the query.

NOTE: This 70% context budget is a default, not a hard limit. It can be overridden per-project in `insightiq.yaml`, though MQS's internal guidance recommends against raising it above 80%, since leaving too little room for the query and response can cause the generation model to produce truncated or malformed answers.

5.3 Citation Formatting

InsightIQ instructs the generation model to cite the source and page number of any chunk it draws on, using the `Source: {{ chunk.source }} (page {{ chunk.page }})` format shown in Section 5.1. This is a prompt-level instruction rather than a guarantee — InsightIQ's Evaluation stage (Section 6) includes a specific metric, Citation Accuracy, precisely because generation models do not always follow citation instructions reliably.

6. Evaluation Metrics

InsightIQ tracks four evaluation metrics for a RAG pipeline, computed against a held-out set of question-answer pairs.

Metric	What It Measures
Retrieval Precision	Fraction of retrieved chunks that are actually relevant to the query
Retrieval Recall	Fraction of all relevant chunks that were successfully retrieved
Answer Faithfulness	Whether the generated answer is fully supported by the retrieved context
Citation Accuracy	Whether cited sources and page numbers actually contain the claimed information

6.1 Why Faithfulness and Accuracy Are Tracked Separately

A generated answer can be faithful to the retrieved context (nothing in the answer contradicts or goes beyond what was retrieved) while still citing the wrong source for a specific claim, if the generation model draws the correct fact from one retrieved chunk but attributes it to another. This is why InsightIQ tracks Answer Faithfulness and Citation Accuracy as two separate metrics rather than a single combined score — a pipeline can score well on one and poorly on the other, and the appropriate fix differs depending on which is failing.

7. Parsing Considerations

Parsing quality has an outsized effect on every downstream stage in this playbook, which is why this document — unlike Documents 1 and 2 in this benchmark suite — deliberately includes figures, UML placeholders, cross-references, and appendices: these are exactly the structures a weak parser tends to mishandle.

7.1 Figure and Caption Association

A parser must correctly associate a figure caption (such as "Figure 3: Ranking comparison...") with the placeholder or image it describes, and must not treat the caption as an unrelated, free-floating sentence. Incorrect association can cause a chunk to contain a caption with no context, or context with no caption, either of which degrades retrieval for any query about that figure.

7.2 Cross-Reference Resolution

This playbook contains numerous cross-references — "see Figure 3," "see Section 8," "see Appendix A" — that a human reader resolves automatically but that a naive chunker may split away from their relevant context, or that a retriever may fail to expand into the section they actually point to. A RAG system that can resolve "see Appendix A" into the actual glossary content in Appendix A, rather than treating it as an unresolvable dead-end phrase, demonstrates meaningfully more sophisticated retrieval than one that cannot.

7.3 Appendix Boundary Detection

Appendices (Appendix A and Appendix B in this document) sit outside the numbered section sequence and can confuse parsers that rely purely on heading-number patterns (1, 2, 3...) to detect section boundaries. A parser that fails to recognize "Appendix A" as a top-level heading equivalent to "Section 7" may merge appendix content into the preceding numbered section, or fail to chunk it as a distinct unit at all.

8. Deployment Patterns

InsightIQ supports two deployment patterns.

Pattern	Description	Re-Ranking Default	Typical Latency Budget
On-Prem	Runs entirely within MQS's own infrastructure, using ClusterGrid for embedding and retrieval workloads	Enabled	Relaxed (internal tools)
InsightIQ Cloud	Managed, multi-tenant deployment hosted by MQS for external customers	Disabled by default	Strict (customer-facing)

On-Prem deployments, run internally at MQS on ClusterGrid, default to enabling the Re-Ranking pass described in Section 4.5, since internal tooling generally tolerates higher latency in exchange for better answer quality. InsightIQ Cloud deployments default to disabling Re-Ranking, since customer-facing latency budgets are typically stricter, though any customer can enable it explicitly in their project configuration if answer quality matters more to them than response time.

9. Case Study: Documentation Search

This section walks through how InsightIQ indexes and serves queries against the very benchmark suite this playbook belongs to, as a concrete illustration of the concepts covered in Sections 3 through 8.

The MQS documentation corpus — the FalconTest guide, the APIVerify handbook, this playbook, and the Engineering Standards Manual described elsewhere in this suite — is indexed using VectorCore-L (Section 3.2) at a chunk size of 550 characters (within the range recommended in Section 3.4), using Hybrid Retrieval (Section 4.3) at the default `hybrid_alpha` of 0.7, with Re-Ranking enabled, consistent with the On-Prem deployment pattern described in Section 8.

A query such as "which cluster runs FalconTest jobs" is deliberately ambiguous in exactly the way Section 4.4's Keyword Retrieval strength is meant to help with: the literal term "cluster" appears in multiple documents referring to multiple distinct entities (Execution Cluster, Reporting Cluster, and ClusterGrid itself), and Hybrid Retrieval's keyword component helps surface all of them for the Re-Ranking pass to then disambiguate using fuller semantic context.

10. Best Practices

- Always match the embedding model between index time and query time; InsightIQ enforces this, but understanding why (Section 3.3) helps when designing multi-project setups.
- Default to Hybrid Retrieval at `hybrid_alpha` 0.7 and tune only after establishing an Evaluation Metrics baseline (Section 6).
- Enable Re-Ranking for On-Prem, internal-tooling deployments; leave it disabled by default for latency-sensitive InsightIQ Cloud deployments unless a customer explicitly needs higher accuracy.
- Track Answer Faithfulness and Citation Accuracy separately — treating them as one combined score hides which part of the pipeline actually needs fixing.
- Test parsing quality specifically on documents with figures, cross-references, and appendices, since these structures fail more often and more subtly than plain prose or simple tables.

Appendix A — Extended Glossary

Term	Definition
VectorCore-S	InsightIQ's compact, 384-dimension embedding model, optimized for speed
VectorCore-L	InsightIQ's larger, 1024-dimension embedding model, optimized for accuracy
Dense Retrieval	Retrieval strategy based on cosine similarity between embedding vectors
Keyword Retrieval	Retrieval strategy based on BM25 term matching
Hybrid Retrieval	Retrieval strategy combining Dense and Keyword results via hybrid_alpha
hybrid_alpha	Tunable parameter (0-1) controlling the Dense/Keyword balance in Hybrid Retrieval
Re-Ranking	Optional second-pass scoring of top-K candidates using a smaller, more precise model
Retrieval Precision	Fraction of retrieved chunks that are actually relevant
Retrieval Recall	Fraction of all relevant chunks that were successfully retrieved
Answer Faithfulness	Whether a generated answer is fully supported by retrieved context
Citation Accuracy	Whether cited sources actually contain the claimed information
ModelMismatchError	Error InsightIQ raises when querying an index with a different embedding model than it was built with

Appendix B — Sample End-to-End Configuration

```
project: mqs-docs-search
embedding:
model: vectorcore-l
chunking:
size: 550
overlap: 110
retrieval:
strategy: hybrid
hybrid_alpha: 0.7
top_k: 20
reranking:
enabled: true
top_n: 5
prompt:
context_budget_pct: 70
system_instruction_pct: 10
generation:
model: mqs-gen-1
deployment:
pattern: on_prem
cluster: clustergrid
```

References

- 1 InsightIQ Embedding Model Benchmarks — internal report, MQS AI Platform Team.
- 2 FalconTest Automation Framework Guide — this benchmark suite, Document 1.
- 3 APIVerify Integration Handbook — this benchmark suite, Document 2.
- 4 Engineering Standards Manual — this benchmark suite, Document 4.

Questions to Try

The following questions are provided to help you explore this document using your own RAG pipeline. No answers are provided here — try retrieving and generating answers yourself.

- 1 What is InsightIQ used for at Meridian Quality Systems?
- 2 Why does InsightIQ enforce embedding model consistency between index time and query time?
- 3 Compare VectorCore-S and VectorCore-L.
- 4 Which retrieval strategy and configuration would you recommend for a new production deployment, and why?
- 5 What is the default value of `hybrid_alpha` in InsightIQ's Hybrid Retrieval?
- 6 Which "Cluster" runs FalconTest jobs, according to the Case Study in Section 9?
- 7 Does InsightIQ support embedding models other than VectorCore-S and VectorCore-L?
- 8 What is the recommended chunk size range for VectorCore-L, according to Section 3.4?
- 9 What is the difference between Answer Faithfulness and Citation Accuracy?
- 10 Should Re-Ranking be enabled for an InsightIQ Cloud deployment? What does the document say determines this?